Piedra, Papel o Tijera con Persistencia de Datos - Manual del Programador

Luca Di Guglielmo Villa
E.E.S.T N°1 de Vicente López
Curso: Laboratorio de Programación 5°3°
Profesor: Ganduglia Yamil & York Elias Mansilla Muñoz
Fecha: 15/10/2026

Introducción

Este manual busca documentar la arquitectura, diseño e implementación del programa trabajado en la actividad 7, “Piedra, Papel o Tijera”. Este programa un sistema de persistencia de datos mediante la libreria ‘fstream’

Desarrollo

1. Arquitectura del Sistema

El programa sigue una arquitectura modular basada en funciones, con una estructura de datos (struct) que agrupa los atributos de cada partida. Componentes principales:

- **struct Partida** - Datos de cada partida
- **jugarPartida()** - Procesa la jugada contra la IA
- **guardarEnDisco()** - Persiste la partida en archivo
- **leerHistorialDisco()** - Recupera y muestra el historial

2. Estructura de Datos

struct Partida { int idPartida; string jugador; string eleccionJugador; string eleccionIA; string resultado; }

```
// 1. Estructura de datos para representar una partida
struct Partida {
    int idPartida;
    string jugador;
    string eleccionJugador;
    string eleccionIA;
    string resultado;
};
```

Se guarda toda la estructura del programa en una misma entidad dentro del código, volviendo más coherente, en esta estructura se guarda el ID (N°) de partida, el jugador / identificador del jugador, la elección del jugador, la elección generada por la IA y el resultado (Jugador ganó, perdió o empató).

3. Manejo de Archivos

El programa realiza la escritura con *ofstream* junto a *ios::app*, este último haciendo que el archivo agregue contenido al final. La lectura, en cambio, se hace con *ifstream*.

```
// ios::app permite agregar al final sin borrar partidas anteriores
ofstream archivo("src/historial_partidas.txt", ios::app);
```

```
// Función para LEER el historial persistido desde el archivo
void leerHistorialDisco() {
    ifstream archivo("src/historial_partidas.txt");
```

4. Manejo de Buffer de entrada

El uso de cin.ignore() se usa antes de getline (usado para recibir entrada de datos tipo string), para eliminar los saltos de línea del output (/n)

```
cout << "\n--- NUEVA PARTIDA #" << p.idPartida << " ---" << endl;
cout << "Ingrese su Nombre / Nickname: ";
cin.ignore(); // Limpieza de búfer
getline(cin, p.jugador);
```

5. Lógica del juego

Se genera una salida random para la IA, (rand() % 3 + 1), para ganar, se debe vencer a la IA en un juego de piedra papel o tijera, (Piedra>Tijera, Papel>Piedra, Tijera>Papel)

6. Escritura y Lectura en Disco

El programa utiliza memoria de Disco para guardar información incluso cuando se cierra el programa. Ambas se hacen en un archivo .txt, creado por el programa, el cual guarda partidas pasadas (utilizando ofstream) y luego, el programa lee este archivo (utilizando ifstream), para mostrar los resultados.

```
void guardarEnDisco(const Partida& p) {
    // ios::app permite agregar al final sin borrar partidas anteriores
    ofstream archivo("src/historial_partidas.txt", ios::app);

    if (!archivo.is_open()) {
        cerr << "[ERROR] No se pudo abrir el archivo para guardar la partida." << endl;
        return;
    }

    // Guardado de datos separados por comas (Formato CSV)
    archivo << p.idPartida << ","
            << p.jugador << ","
            << p.eleccionJugador << ","
            << p.eleccionIA << ","
            << p.resultado << endl;

    archivo.close();
    cout << "[DISCO] Partida resguardada con exito en 'historial_partidas.txt'." << endl;
}
```

```
void leerHistorialDisco() {
    ifstream archivo("src/historial_partidas.txt");

    if (!archivo.is_open()) {
        cout << "\n[AVISO] Aun no existen partidas guardadas en disco." << endl;
        return;
    }

    cout << "\n=====================================================" << endl;
    cout << " HISTORIAL DE PARTIDAS RECUPERADO DESDE DISCO     " << endl;
    cout << "=====================================================" << endl;

    string idStr, jugador, elJug, elIA, resultado;
    int total = 0;

    // Lectura secuencial delimitada por comas
    while (getline(archivo, idStr, ',') &&
           getline(archivo, jugador, ',') &&
           getline(archivo, elJug, ',') &&
           getline(archivo, elIA, ',') &&
           getline(archivo, resultado)) {

        total++;
        cout << "Partida #" << idStr
             << " | Jugador: " << jugador
             << " | " << elJug << " vs " << elIA
             << " | Resultado: " << resultado << endl;
    }

    archivo.close();

    if (total == 0) {
        cout << "[AVISO] El archivo de historial esta vacio." << endl;
    } else {
        cout << "\n[SISTEMA] Total de registros leidos desde disco: " << total << endl;
    }
    cout << "=====================================================" << endl;
}
```

Conclusión

En este programa se pone en desarrollo los conceptos fundamentales de la persistencia de datos en C + +, en este manual se explico estos conceptos y otros detalles lógicos importantes del programa como el manejo del buffer de entrada, manejo de archivo y la estructura de datos.

Referencias:

Luca Di Guglielmo

Github: https://github.com/specIux/actividad-7-LPR

Laboratorio de Programación 5°3°

Actividad: 📘📺🛠️🖌️ ACTIVIDAD 6: TALLER PRÁCTICO DE REPASO Y CONSOLIDACIÓ…